EXCEPTION HANDLING

CSC103 Programming Fundamentals

TODAY'S LECTURE

- Exceptions and exception handling
- `Try-throw-catch` template
- Exception classes
- Exception handling model
  - Declaring exceptions
  - Throwing exceptions
  - Catching exceptions
- The `finally` clause
- Rethrowing exceptions
- Chained exceptions

INTRODUCTION

- *Exception handling enables a program to deal with exceptional situations and continue its normal execution*
- Runtime errors occur while a program is running (JVM detects an operation that is impossible to carry out)
- Examples:
  - Accessing an array using an index that is out of bounds (`ArrayIndexOutOfBoundsException`)
  - Entering a double value when your program expects an integer (`InputMismatchException`)

EXCEPTIONS

- In Java, runtime errors are THROWN as *exceptions*
- Exception: an object that represents an error or a condition that prevents execution from proceeding normally
  - If not handled, the program will terminate abnormally
- Advantages:
  - The core advantage of exception handling is to maintain the normal flow of the application.
  - Enables a method to throw an exception to its caller, enabling the caller to handle the exception
  - Separates the detection of an error from the handling of an error

Example

```
1.statement 1;
2.statement 2;
3.statement 3;
4.statement 4;
5.statement 5;//exception occurs
6.statement 6;
7.statement 7;
8.statement 8;
9.statement 9;
10.statement 10;
```

JAVA EXCEPTION HANDLING EXAMPLE AND COMMON SCENARIOS

```
public class JavaExceptionExample{
  public static void main(String args[]){
   try{
     //code that may raise exception
     int data=100/0;
   }catch(ArithmeticException e){
        System.out.println(e);
        }
   //rest code of the program
   System.out.println("rest of the code...");
  }
}
```

Exception in thread main java.lang.ArithmeticException:/ by zero
rest of the code...

- A scenario where ArithmeticException occurs:

```
int a = 100/0;
```

- A scenario where NullPointerException occurs:

```
String s=null;
System.out.println(s.length())
```

- A scenario where NumberFormatException occurs

```
String s="abc";
int i=Integer.parseInt(s);//NumberFormatException
```

- A scenario where ArrayIndexOutOfBoundsException occurs

```
int a[]=new int[5];
a[10]=50; //ArrayIndexOutOfBoundsException
```

EXCEPTION HANDLING

- Exceptions are thrown from a method
- The caller of the method can catch and handle the exception

```
1  import java.util.Scanner;
2
3  public class Quotient {
4    public static void main(String[] args) {
5      Scanner input = new Scanner(System.in);
6
7      // Prompt the user to enter two integers
8      System.out.print("Enter two integers: ");
9      int number1 = input.nextInt();
10     int number2 = input.nextInt();
11
12     System.out.println(number1 + " / " + number2 + " is " +
13       (number1 / number2));
14   }
15 }
```

```
Enter two integers: 5 2 Enter
5 / 2 is 2
```

```
Enter two integers: 3 0 Enter
Exception in thread "main" java.lang.ArithmeticException: / by zero
at Quotient.main(Quotient.java:11)
```

HANDLING THE EXCEPTION USING IF STATEMENT

```
1  import java.util.Scanner;
2
3  public class QuotientWithIf {
4    public static void main(String[] args) {
5      Scanner input = new Scanner(System.in);
6
7      // Prompt the user to enter two integers
8      System.out.print("Enter two integers: ");
9      int number1 = input.nextInt();
10     int number2 = input.nextInt();
11
12     if (number2 != 0)
13       System.out.println(number1 + " / " + number2
14         + " is " + (number1 / number2));
15     else
16       System.out.println("Divisor cannot be zero ");
17   }
18 }
```

```
Enter two integers: 5 0 Enter
Divisor cannot be zero
```

THE PROBLEM

```
1  import java.util.Scanner;
2
3  public class QuotientWithMethod {
4    public static int quotient(int number1, int number2) {
5      if (number2 == 0) {
6        System.out.println("Divisor cannot be zero");
7        System.exit(1);
8      }
9
10     return number1 / number2;
11   }
12
13   public static void main(String[] args) {
14     Scanner input = new Scanner(System.in);
15
16     // Prompt the user to enter two integers
17     System.out.print("Enter two integers: ");
18     int number1 = input.nextInt();
19     int number2 = input.nextInt();
20
21     int result = quotient(number1, number2);
22     System.out.println(number1 + " / " + number2 + " is "
23       + result);
24   }
25 }
```

```
Enter two integers: 5 3 Enter
5 / 3 is 1
```

```
Enter two integers: 5 0 Enter
Divisor cannot be zero
```

```
1  import java.util.Scanner;
2
3  public class QuotientWithException {
4    public static int quotient(int number1, int number2) {
5      if (number2 == 0)
6        throw new ArithmeticException("Divisor cannot be zero");
7
8      return number1 / number2;
9    }
10
11   public static void main(String[] args) {
12     Scanner input = new Scanner(System.in);
13
14     // Prompt the user to enter two integers
15     System.out.print("Enter two integers: ");
16     int number1 = input.nextInt();
17     int number2 = input.nextInt();
18
19     try {
20       int result = quotient(number1, number2);
21       System.out.println(number1 + " / " + number2 + " is "
22         + result);
23     }
24     catch (ArithmeticException ex) {
25       System.out.println("Exception: an integer " +
26         "cannot be divided by zero ");
27     }
28
29     System.out.println("Execution continues ...");
30   }
31 }
```

If an ArithmeticException occurs

```
Enter two integers: 5 3 Enter
5 / 3 is 1
Execution continues ...
```

```
Enter two integers: 5 0 Enter
Exception: an integer cannot be divided by zero
Execution continues ...
```

THE SOLUTION

EXCEPTION HANDLING

- The value thrown in the example, `new ArithmeticException("Divisor cannot be zero")`, is an **exception**
- The execution of a throw statement is called **throwing an exception**
- In this example, the exception class is `java.lang.ArithmeticException`
- The constructor `ArithmeticException(str)` is invoked to construct an exception object, where `str` is a message that describes the exception
- When an exception is thrown, the normal flow execution is interrupted
  - **Throwing**: passing an exception from one place to another
  - Analogous to a method call (but it calls the `catch` block)
- The method call is contained in a `try` and `catch` block
- `Try` **block:** code that is executed in normal circumstances
- `Catch` **block:** code that catches the exception; handles the exception
  - Analogous to method definition with a parameter that matches the type of the value being thrown

EXCEPTION HANDLING (CONTD.)

- `Catch-block parameter`: the identifier `ex` in the `catch` block header
  - The type (e.g. `ArithmeticException`) preceding `ex` specifies what kind of exception the `catch` block can catch
  - The thrown value can be accessed from this parameter in the body of `catch` block
- Template of `try-throw-catch`

```
try {
  Code to run;
  A statement or a method that may throw an exception;
  More code to run;
}
catch (type ex) {
  Code to process the exception;
}
```

Throwing an exception:

- A `throw` statement in a `try` block
- Method invocation

EXAMPLE: LIBRARY METHOD THROWING EXCEPTION

```
1  import java.util.*;
2
3  public class InputMismatchExceptionDemo {
4    public static void main(String[] args) {
5      Scanner input = new Scanner(System.in);
6      boolean continueInput = true;
7
8      do {
9        try {
10         System.out.print("Enter an integer: ");
11         int number = input.nextInt();
12
13         // Display the result
14         System.out.println(
15           "The number entered is " + number);
16
17         continueInput = false;
18       }
19       catch (InputMismatchException ex) {
20         System.out.println("Try again. (" +
21           "Incorrect input: an integer is required)");
22         input.nextLine(); // Discard input
23       }
24     } while (continueInput);
25   }
26 }
```

If an InputMismatchException occurs

```
Enter an integer: 3.5 Enter
Try again. (Incorrect input: an integer is required)
Enter an integer: 4 Enter
The number entered is 4
```

EXAMPLES

```
System.out.println(1 / 0);
System.out.println(1.0 / 0);
```

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at exceptionhandling.ExceptionHandling.main(ExceptionHandling.java:19)
Java Result: 1
```

```
Infinity
```

```
System.out.println(0.0 / 0);
```

```
NaN
```

```
public class Test {
  public static void main(String[] args) {
    try {
      int value = 30;
      if (value < 40)
        throw new Exception("value is too small");
    }
    catch (Exception ex) {
      System.out.println(ex.getMessage());
    }
    System.out.println("Continue after the catch block");
  }
}
```

```
value is too small
Continue after the catch block
```

```
int value = 50;
```

```
Continue after the catch block
```

TRY – CATCH – FINALLY SYNTAX

```
try{ … }
catch(){
}
finally{
}
```

FOUR POSSIBILITIES OF EXCEPTION

- Default Throw and Default Catch
- Default Throw and our Catch
- Our Throw and Default Catch
- Our Throw and Our Catch

DEFAULT THROW AND DEFAULT CATCH

```
int[ ] abc = new int[5];
System.out.println("First Line"+abc[5]); //ArrayIndexOutOfBoundsException
System.out.println(2/0);//ArithmeticException
System.out.println("Last Line");
```

DEFAULT THROW AND OUR CATCH

- catch can be more than one associated with one try
- Each try must have one catch(){…}

```
try{
        String st = null; //NullPointerException
        int[] abc = null; //NullPointerException
        System.out.println("First Line"+st.length());
        System.out.println("Last Line after exception");
        //System.out.println(20/0);//ArithmeticException: Java by default make its object
    }
    catch(NullPointerException ex){
        System.out.println("!!!No Reference to Object!!!");
    }
```

OUR THROW AND DEFAULT CATCH

- Explicit Throw: A program can explicitly throw an Exception using the "throw" statement besides the implicit exception thrown
- Syntax: throw <throwableInstance>
- Example: throw new ArithmeticException("Division by Zero")

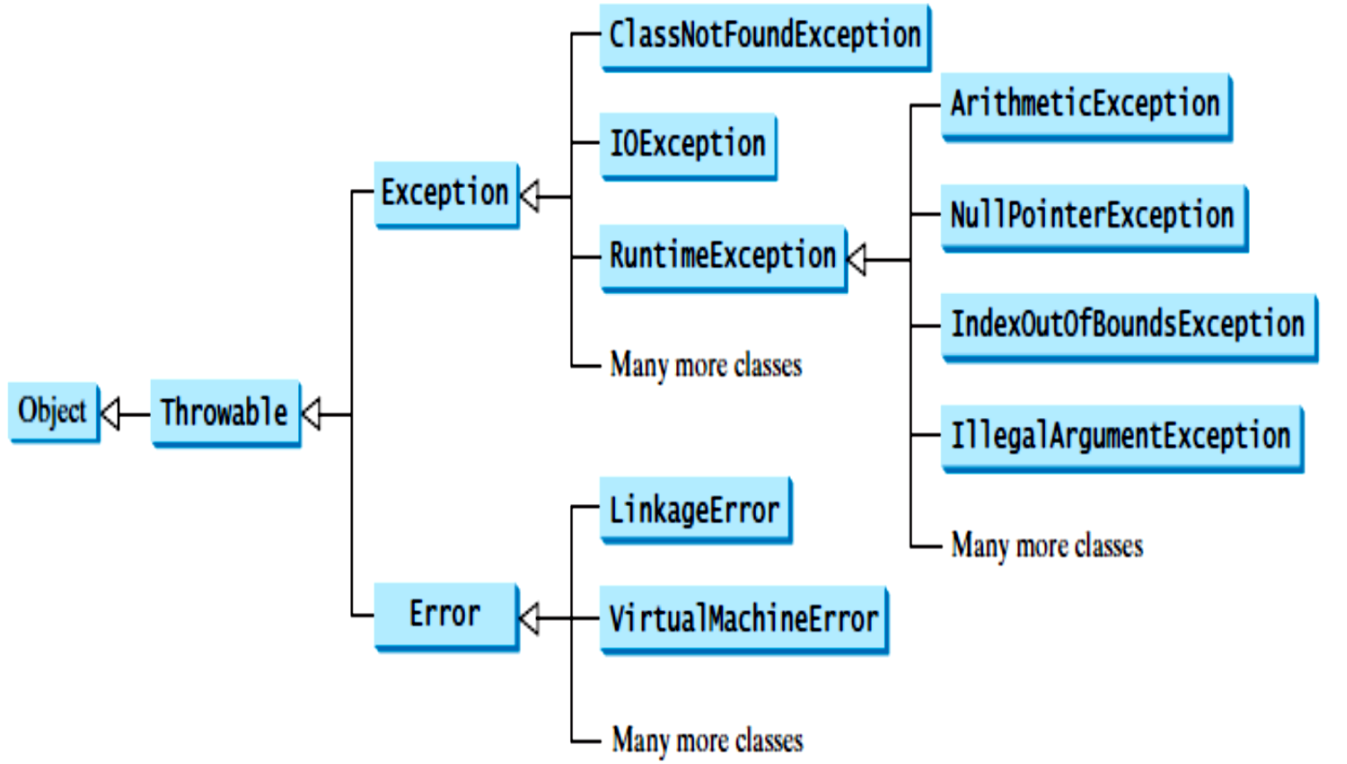
```
int balance = 1000;
int withdrawlAmount = 3000;
if(balance < withdrawlAmount){
                        throw new ArithmeticException("<<<Insufficient Balance...>>>");
                    }
balance = balance - withdrawlAmount;
System.out.println("Sucessfully Completed");
```

OUR THROW AND OUR CATCH

```
try{
        int balance = 1000;
        int withdrawlAmount = 3000;
        if(balance < withdrawlAmount){
            throw new ArithmeticException("Insufficient Balance...");
        }
        balance = balance - withdrawlAmount;
        System.out.println("Sucessfully Completed");
    }
    catch(ArithmeticException e){
        System.out.println("Exception: "+ e.getMessage());
    }
```

EXCEPTION TYPES

- Exception are objects; Objects are defined using classes
- Many predefined exception classes in the Java API; one can define one's own exception classes
- The root class for exceptions is `java.lang.Throwable`



EXCEPTION CLASSES

System errors

- Thrown by JVM
- Represented in `Error` class

Describes internal system errors

Exceptions

- Caught & handled by your program
- Represented in `Exception` class

Describes errors caused by your program & by external circumstances

Runtime Exceptions

- Caught & handled by your program
- Represented in `RuntimeException` class

Describes programming errors

Class	*Reasons for Exception*
`LinkageError`	A class has some dependency on another class, but the latter class has changed incompatibly after the compilation of the former class.
`VirtualMachineError`	The JVM is broken or has run out of the resources it needs in order to continue operating.

Class	*Reasons for Exception*
`ArithmeticException`	Dividing an integer by zero. Note that floating-point arithmetic does not throw exceptions (see Appendix E, Special Floating-Point Values).
`NullPointerException`	Attempt to access an object through a `null` reference variable.
`IndexOutOfBoundsException`	Index to an array is out of range.
`IllegalArgumentException`	A method is passed an argument that is illegal or inappropriate.

Class	*Reasons for Exception*
`ClassNotFoundException`	Attempt to use a class that does not exist. This exception would occur, for example, if you tried to run a nonexistent class using the **java** command, or if your program were composed of, say, three class files, only two of which could be found.
`IOException`	Related to input/output operations, such as invalid input, reading past the end of a file, and opening a nonexistent file. Examples of subclasses of `IOException` are `InterruptedIOException`, `EOFException` (EOF is short for End of File), and `FileNotFoundException`.

UN/CHECKED EXCEPTIONS

- Unchecked exceptions: `RuntimeException, Error`
  - Programming logic errors that are unrecoverable; not mandatory to catch or declare
  - `NullPointerException`: if you access an object through a reference variable before an object is assigned to it
  - `IndexOutOfBoundsException`: if you access an element in an array outside the bounds of array
- Checked exceptions: all other exceptions
  - Compiler forces the programmer to check
  - Dealt in
    - `Try-catch` block
    - Declared in the method header

```java
public class Test {
  public static void main(String[] args) {
    String s = "abc";
    System.out.println(s.charAt(3));
  }
}
```

```java
public class Test {
  public static void main(String[] args) {
    Object o = null;
    System.out.println(o.toString());
  }
}
```

```java
public class Test {
  public static void main(String[] args) {
    System.out.println(1 / 0);
  }
}
```

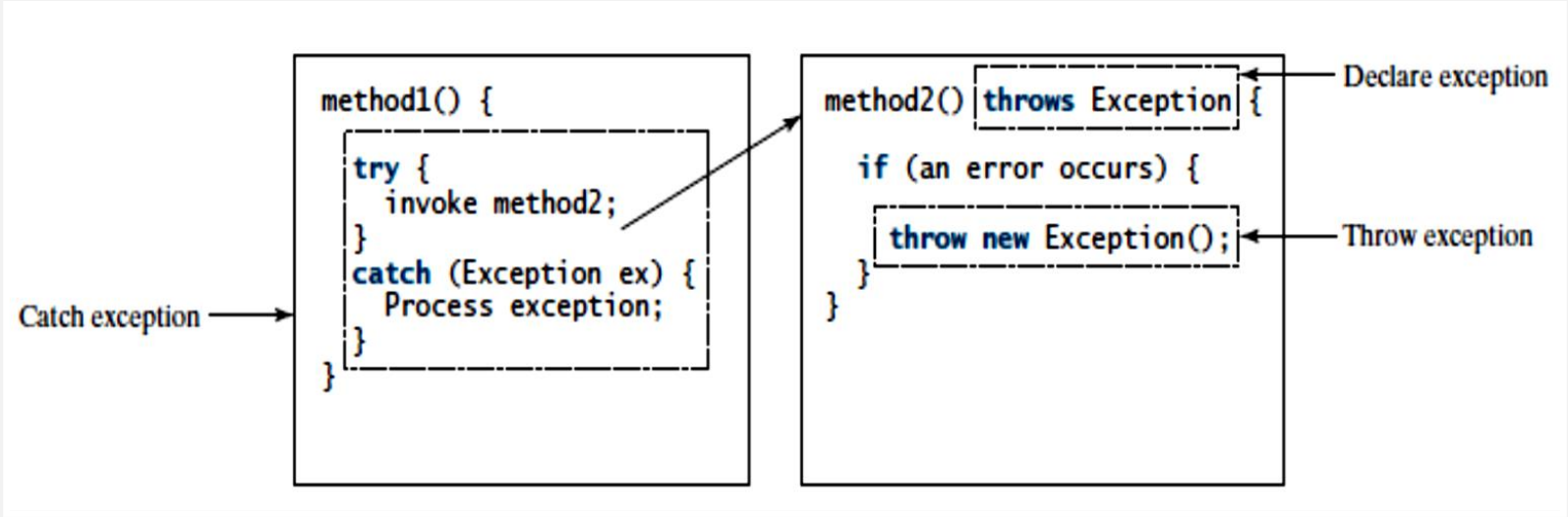
```java
public class Test {
  public static void main(String[] args) {
    Object o = new Object();
    String d = (String)o;
  }
}
```

```java
public class Test {
  public static void main(String[] args) {
    int[] list = new int[5];
    System.out.println(list[5]);
  }
}
```

```java
public class Test {
  public static void main(String[] args) {
    System.out.println(1.0 / 0);
  }
}
```

MORE ON EXCEPTION HANDLING

- A handler for an exception is found by propagating the exception backward through a chain of method calls, starting from the current method
- Exception-handling model in Java
  - Declaring an exception
  - Throwing an exception
  - Catching an exception



DECLARING EXCEPTIONS

- Every method must state the types of **checked** exceptions it might throw. This is known as declaring exceptions.
- **Unchecked** exceptions need not be declared explicitly in the method
- Use keyword `throws` in the method header

```
public void myMethod() throws IOException
```

```
public void myMethod()
  throws Exception1, Exception2, ..., ExceptionN
```

THROWING EXCEPTIONS

- A program that detects an error can create an instance/object of an appropriate exception type and throw it – throwing exception

```
IllegalArgumentException ex =
  new IllegalArgumentException("Wrong Argument");
throw ex;
```

```
throw new IllegalArgumentException("Wrong Argument");
```

- Each exception class has at least 2 constructors:
- A non-argument constructor
- An argument constructor
  - Exception message can be obtained using `getMessage()`

CATCH/DECLARE CHECKED EXCEPTIONS

```
void p1() {
  try {
    p2();
  }
  catch (IOException ex) {
    ...
  }
}
```

(a) Catch exception

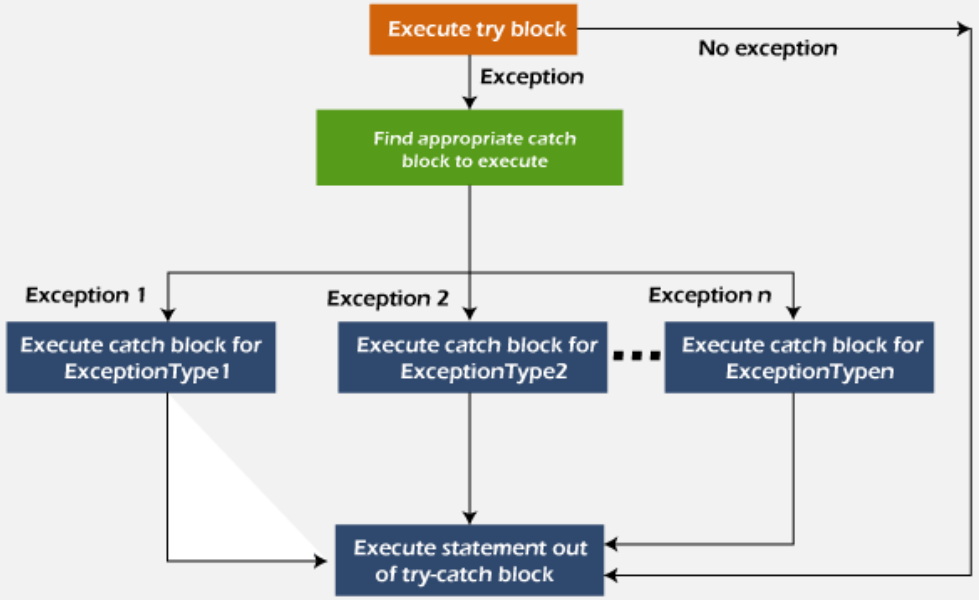
```
void p1() throws IOException {

  p2();

}
```

(b) Throw exception

```
catch (Exception1 | Exception2 | ... | Exceptionk ex) {
  // Same code for handling these exceptions
}
```

JAVA MULTI-CATCH BLOCK

- A try block can be followed by one or more catch blocks
- At a time only one exception occurs and at a time only one catch block is executed
- All catch blocks must be ordered from most specific to most general, i.e. catch for ArithmeticException must come before catch for Exception.



EXAMPLE

```
public class MultipleCatchBlock1 {

   public static void main(String[] args) {

        try{
            int a[]=new int[5];
            a[5]=30/0;
           }
           catch(ArithmeticException e)
             {
              System.out.println("Arithmetic Exception occurs");
             }
           catch(ArrayIndexOutOfBoundsException e)
             {
              System.out.println("ArrayIndexOutOfBounds Exception occurs");
             }
           catch(Exception e)
             {
              System.out.println("Parent Exception occurs");
             }
           System.out.println("rest of the code");
   }
}
```

ORDER OF EXCEPTIONS

- The order in which exceptions are specified in `catch` blocks is important.
- **A compile error**: if a `catch` block for a superclass type appears before a `catch` block for a subclass type

```
try {
  ...
}
catch (Exception ex) {
  ...
}
catch (RuntimeException ex) {
  ...
}
```

(a) Wrong order

```
try {
  ...
}
catch (RuntimeException ex) {
  ...
}
catch (Exception ex) {
  ...
}
```

(b) Correct order

CATCHING EXCEPTIONS

An exception can be caught and handled in a `try-catch` block

In case no exceptions are thrown, `catch` block(s) are skipped

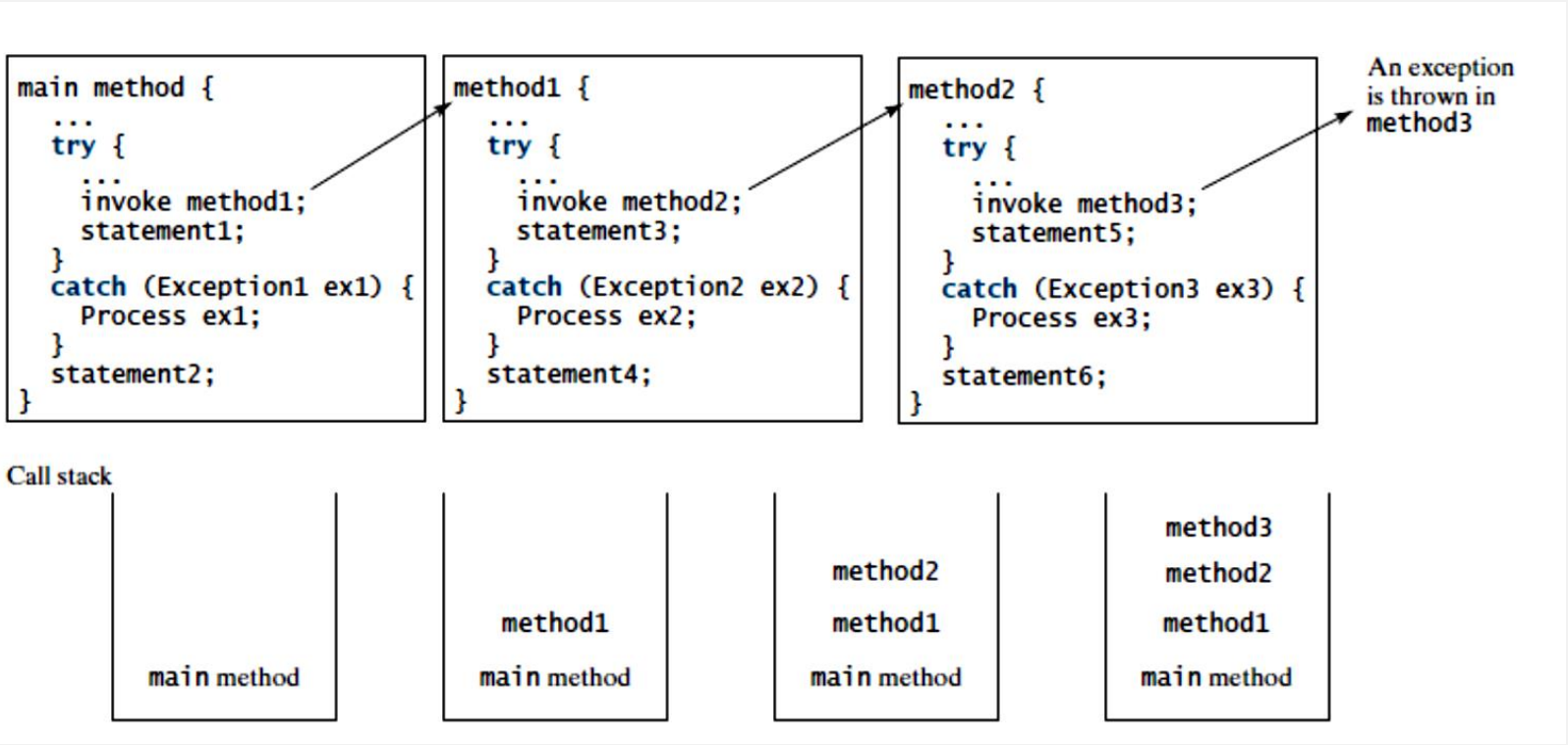
Exception handler: The code that handles the exception

Found by propagating the exception backward through a chain of method calls, starting from the current method

```
try {
  statements; // Statements that may throw exceptions
}
catch (Exception1 exVar1) {
  handler for exception1;
}
catch (Exception2 exVar2) {
  handler for exception2;
}
...
catch (ExceptionN exVarN) {
  handler for exceptionN;
}
```

CATCHING EXCEPTIONS/ EXCEPTION PROPAGATION

- An exception is first thrown from the top of the stack
- If it is not caught, it drops down the call stack to the previous method.
- If not caught there, the exception again drops down to the previous method, and so on until they are caught or until they reach the very bottom of the call stack. This is called exception propagation

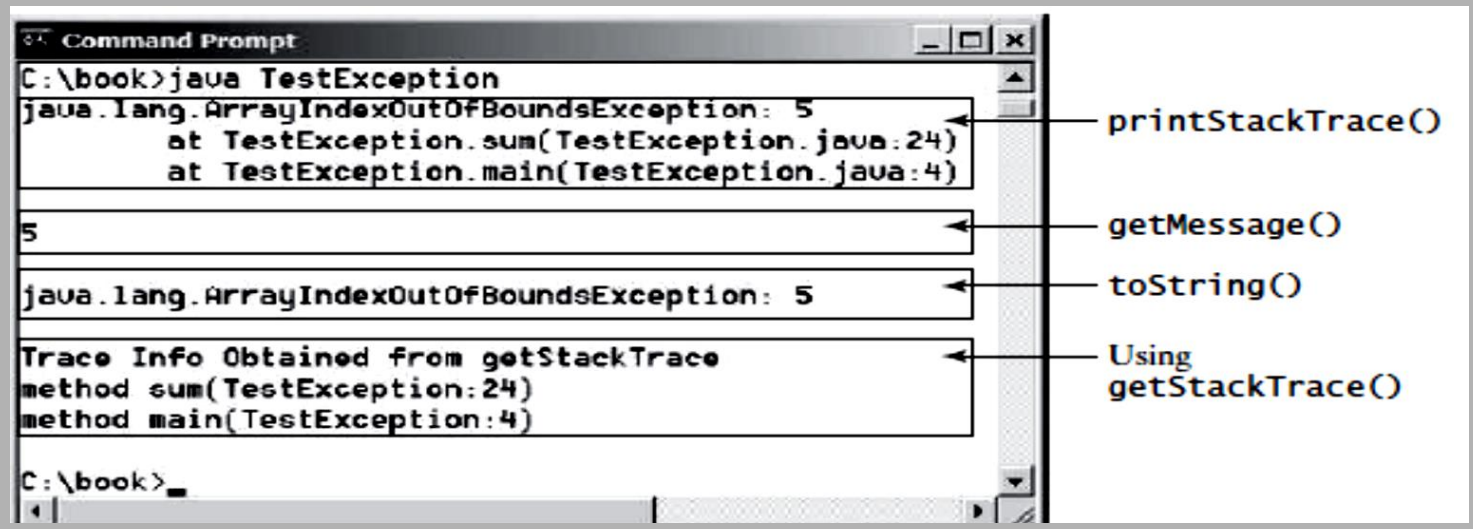


CATCHING EXCEPTIONS/ EXCEPTION PROPAGATION

- If the exception type is Exception3, it is caught by the catch block for handling exception ex3 in method2. statement5 is skipped, and statement6 is executed.
- If the exception type is Exception2, method2 is aborted, the control is returned to method1, and the exception is caught by the catch block for handling exception ex2 in method1. statement3 is skipped, and statement4 is executed.
- If the exception type is Exception1, method1 is aborted, the control is returned to the main method, and the exception is caught by the catch block for handling exception ex1 in the main method. statement1 is skipped, and statement2 is executed.
- If the exception type is not caught in method2, method1, or main, the program terminates, and statement1 and statement2 are not executed.

GETTING INFORMATION FROM EXCEPTIONS

java.lang.Throwable	
+getMessage(): String	Returns the message that describes this exception object.
+toString(): String	Returns the concatenation of three strings: (1) the full name of the exception class; (2) ":" (a colon and a space); (3) the `getMessage()` method.
+printStackTrace(): void	Prints the `Throwable` object and its call stack trace information on the console.
+getStackTrace(): StackTraceElement[]	Returns an array of stack trace elements representing the stack trace pertaining to this exception object.



THE FINALLY CLAUSE

- The `finally` clause is always executed regardless whether an exception has occurred or not
- Three possible cases:
  - No exception in `try` block, `finalStatements` is executed, & the next statement after the `try` statement is executed
  - An exception in `try` block, caught in `catch` block, the rest of the statements in `try` block are skipped, `catch` block is executed, & `finally` clause is executed. The next statement after the `try` statement is executed
  - An exception in `try` block, not caught in `catch` block, the other statements in the `try` block are skipped, the `finally` clause is executed, & the exception is passed to the caller method

```
try {
  statements;
}
catch (TheException ex) {
  handling ex;
}

finally {
  finalStatements;
}
```

```
try {
    statement1;
    statement2;
    statement3;
}
catch (Exception1 ex1) {
    statement4;
}
finally {
    statement5;
}
statement6;
```

WHEN TO USE EXCEPTIONS

- A method should throw an exception if the error needs to be handled by its caller
- Exception handling separates error-handling code from normal programming tasks
- Makes program easier to read and to modify
- **Drawback**: more time, more resources
- Simple errors that may occur in individual methods are best handled without throwing exceptions
- Use `try-catch` block while dealing with unexpected error conditions

RETHROWING EXCEPTIONS

- Rethrowing exception:
  - If the handler cannot process the exception
  - If the caller must be notified of the exception

```
try {
  statements;
}
catch (TheException ex) {
  perform operations before exits;
  throw ex;
}
```

EXAMPLE

```
public class Demo {
   public static void test1() {
      System.out.println("The Exception in test1() method");
      throw new ArithmeticException("(ArithmeticException) thrown from test1() method");
   }
   public static void test2(){
      try {
         test1();
      } catch(ArithmeticException e) {
         System.out.println("Inside test2() method");
         throw e;
      }
   }
   public static void main(String[] args)  {
      try {
         test2();
      } catch(ArithmeticException e) {
         System.out.println("Caught in main  " +e.getMessage());
         //e.printStackTrace();
      }
   }
}
```

```
The Exception in test1() method
Inside test2() method
Caught in main  (ArithmeticException)
thrown from test1() method
```

CHAINED EXCEPTIONS

Throwing an exception along with another exception forms a chained exception

```
1  public class ChainedExceptionDemo {
2    public static void main(String[] args) {
3      try {
4        method1();
5      }
6      catch (Exception ex) {
7        ex.printStackTrace();
8      }
9    }
10
11   public static void method1() throws Exception {
12     try {
13       method2();
14     }
15     catch (Exception ex) {
16       throw new Exception("New info from method1", ex);
17     }
18   }
19
20   public static void method2() throws Exception {
21     throw new Exception("New info from method2");
22   }
23 }
```

```
java.lang.Exception: New info from method1
  at ChainedExceptionDemo.method1(ChainedExceptionDemo.java:16)
  at ChainedExceptionDemo.main(ChainedExceptionDemo.java:4)
Caused by: java.lang.Exception: New info from method2
  at ChainedExceptionDemo.method2(ChainedExceptionDemo.java:21)
  at ChainedExceptionDemo.method1(ChainedExceptionDemo.java:13)
  ... 1 more
```

RECAP

- Exception handling
- Exception handling model
  - Declaring, throwing, & catching exceptions
- Exception types
- The `finally` clause
- Rethrowing exceptions
- Chained exceptions